

Trabalho de Algoritmos

Nome: Renan Croffi · Matéria: Algoritmos e Lógica de Programação

Professor: Eric · 26 de ago. de 2026

Ex 1 — algoritmo "Soma, Subtração, Multiplicação e Divisão"

```
algoritmo "Soma, Subtração, Multiplicação e Divisão"

var
  n1, n2, soma, sub, division, mult: real

inicio

  escreval("Digite o primeiro numero: ")
  leia(n1)
  escreval("Digite o segundo digito: ")
  leia(n2)
  soma <- n1 + n2
  sub <- n1 - n2
  division <- n1 / n2
  mult <- n1 * n2

  escreval("Soma:",soma)
  escreval("Subtração:",sub)
  escreval("Multiplicação:",mult)
  escreval("Divisão:",division)

finalgoritmo
```

Teste de mesa 1 — entradas: n1 = 20, n2 = 4

Instrução	n1	n2	soma	sub	division	mult	Saída
escreval("Digite o primeiro numero: ")	—	—	—	—	—	—	Digite o primeiro numero:
leia(n1)	20	—	—	—	—	—	
escreval("Digite o segundo digito: ")	20	—	—	—	—	—	Digite o segundo digito:
leia(n2)	20	4	—	—	—	—	
soma <- n1 + n2	20	4	24	—	—	—	
sub <- n1 - n2	20	4	24	16	—	—	
division <- n1 / n2	20	4	24	16	5	—	
mult <- n1 * n2	20	4	24	16	5	80	
escreval("Soma:",soma)	20	4	24	16	5	80	Soma:24
escreval("Subtração:",sub)	20	4	24	16	5	80	Subtração:16
escreval("Multiplicação:",mult)	20	4	24	16	5	80	Multiplicação:80
escreval("Divisão:",division)	20	4	24	16	5	80	Divisão:5

Ex 2 — algoritmo "Área retângulo"

```
algoritmo "Área retângulo"

var
    n1, n2, area : real

inicio

    escreval("Digite a base do retângulo: ")
    leia(n1)
    escreval("Digite a altura: ")
    leia(n2)
    area <- n1 * n2
    escreval("Área do retângulo: ", area)

finalgoritmo
```

Teste de mesa 1 — entradas: base = 8, altura = 5

Instrução	n1	n2	area	Saída
escreval("Digite a base do retângulo: ")	—	—	—	Digite a base do retângulo:
leia(n1)	8	—	—	
escreval("Digite a altura: ")	8	—	—	Digite a altura:
leia(n2)	8	5	—	
area <- n1 * n2	8	5	40	
escreval("Área do retângulo: ", area)	8	5	40	Área do retângulo: 40

Ex 3 — algoritmo "Média"

```
algoritmo "Média"

var
    n1, n2, n3, media : real

inicio
    escreval("Entre a nota 1: ")
    leia(n1)
    escreval("Entre a nota 2: ")
    leia(n2)
    escreval("Entre a nota 3: ")
    leia(n3)
    media <- (n1+n2+n3)/3
    escreval("Média do aluno: ", media)

finalgoritmo
```

Teste de mesa 1 — entradas: n1 = 7, n2 = 5, n3 = 8

Instrução	n1	n2	n3	media	Saída
escreval("Entre a nota 1: ")	—	—	—	—	Entre a nota 1:
leia(n1)	7	—	—	—	
escreval("Entre a nota 2: ")	7	—	—	—	Entre a nota 2:
leia(n2)	7	5	—	—	
escreval("Entre a nota 3: ")	7	5	—	—	Entre a nota 3:
leia(n3)	7	5	8	—	
media <- (n1+n2+n3)/3	7	5	8	6,666667	
escreval("Média do aluno: ", media)	7	5	8	6,666667	Média do aluno: 6,666667

Ex 4 — algoritmo "Antecessor e Sucessor"

```
algoritmo "Antecessor e Sucessor"

var
    n1, n2, n3: inteiro

inicio
    escreval("Entre um número")
    leia(n1)
    n2 <- n1 -1
    n3 <- n1 + 1
    escreval("O antecessor de ", n1, " é: ", n2)
    escreval("O sucessor de ",n1," é: ", n3)
finalgoritmo
```

Teste de mesa 1 — entradas: n1 = 10

Instrução	n1	n2	n3	Saída
escreval("Entre um número")	—	—	—	Entre um número
leia(n1)	10	—	—	
n2 <- n1 -1	10	9	—	
n3 <- n1 + 1	10	9	11	
escreval("O antecessor de ", n1, " é: ", n2)	10	9	11	O antecessor de 10 é: 9
escreval("O sucessor de ",n1," é: ", n3)	10	9	11	O sucessor de 10 é: 11

Ex 5 — algoritmo "Total da Compra"

```
algoritmo "Total da Compra "

var
    itens: inteiro
    preco, total: real

inicio
    escreval("Entre com a quantidade de itens: ")
    leia(itens)
    escreval("Entre com o valor de cada item: ")
    leia(preco)
    total <- itens * preco
    escreval("O total da compra é: R$ ", total)
finalgoritmo
```

Teste de mesa 1 — entradas: itens = 25, preco = 4

Instrução	itens	preco	total	Saída
escreval("Entre com a quantidade de itens: ")	—	—	—	Entre com a quantidade de itens:
leia(itens)	25	—	—	
escreval("Entre com o valor de cada item: ")	25	—	—	Entre com o valor de cada item:
leia(preco)	25	4	—	
total <- itens * preco	25	4	100	
escreval("O total da compra é: R\$ ", total)	25	4	100	O total da compra é: R\$ 100

Ex 6 — algoritmo "Celsius para Fahrenheit"

```
algoritmo "Celsius para Fahrenheit "  
  
var  
    tempC, tempF : real  
  
inicio  
    escreval("Entre a temperatura em celsius: ")  
    leia(tempC)  
    tempF<- (9/5 * tempC) + 32  
    escreval("A temperatura em Fahrenheit é: ", tempF)  
finalgoritmo
```

Teste de mesa 1 — entradas: tempC = 25

Instrução	tempC	tempF	Saída
escreval("Entre a temperatura em celsius: ")	—	—	Entre a temperatura em celsius:
leia(tempC)	25	—	
tempF <- (9/5 * tempC) + 32	25	77	
escreval("A temperatura em Fahrenheit é: ", tempF)	25	77	A temperatura em Fahrenheit é: 77

Ex 7 — algoritmo "Área do trapézio"

```
algoritmo "Área do trapézio"  
  
var  
    B, bm, h, area: real  
  
inicio  
    escreval("Entre com a base maior(B): ")  
    leia(B)  
    escreval("Entre com a base menor(b): ")  
    leia(bm)  
    escreval("Entre a altura: ")  
    leia(h)  
    area<- ((B + bm)*h)/2  
    escreval("Área do trapézio: ", area)  
finalgoritmo
```

Teste de mesa 1 — entradas: B = 10, bm = 6, h = 4

Instrução	B	bm	h	area	Saída
escreval("Entre com a base maior(B): ")	—	—	—	—	Entre com a base maior(B):
leia(B)	10	—	—	—	
escreval("Entre com a base menor(b): ")	10	—	—	—	Entre com a base menor(b):
leia(bm)	10	6	—	—	
escreval("Entre a altura: ")	10	6	—	—	Entre a altura:
leia(h)	10	6	4	—	
area <- ((B + bm)*h)/2	10	6	4	32	
escreval("Área do trapézio: ", area)	10	6	4	32	Área do trapézio: 32

Ex 8 — algoritmo "Olá"

```
algoritmo "Olá"

var
  nome: string

inicio
  escreval("Entre seu nome: ")
  leia(nome)
  escreva("Olá ",nome)
finalgoritmo
```

Teste de mesa 1 — entradas: nome = "Renan"

Instrução	nome	Saída
escreval("Entre seu nome: ")	—	Entre seu nome:
leia(nome)	Renan	
escreva("Olá ",nome)	Renan	Olá Renan

Ex 9 — algoritmo "Média + aprovação"

```
algoritmo "Média + aprovação"

var
  n1, n2, media: real

inicio
  escreval("Entre a primeira nota: ")
  leia(n1)
  escreval("Entre a segunda nota: ")
  leia(n2)
  media<- (n1 + n2)/2
  se media >= 6 entao
    escreval("Aluno aprovado!")
  senao
    escreval("Aluno reprovado!")
  fimse
finalgoritmo
```

Teste de mesa 1 — entradas: n1 = 7, n2 = 5

Instrução	n1	n2	media	Condição	Saída
escreval("Entre a primeira nota: ")	—	—	—		Entre a primeira nota:
leia(n1)	7	—	—		
escreval("Entre a segunda nota: ")	7	—	—		Entre a segunda nota:
leia(n2)	7	5	—		
media <- (n1 + n2)/2	7	5	6		
se media >= 6 entao	7	5	6	6 >= 6 → V	
escreval("Aluno aprovado!")	7	5	6		Aluno aprovado!

Teste de mesa 2 — entradas: n1 = 4, n2 = 5

Instrução	n1	n2	media	Condição	Saída
escreval("Entre a primeira nota: ")	—	—	—		Entre a primeira nota:
leia(n1)	4	—	—		
escreval("Entre a segunda nota: ")	4	—	—		Entre a segunda nota:
leia(n2)	4	5	—		
media <- (n1 + n2)/2	4	5	4,5		
se media >= 6 entao	4	5	4,5	4,5 >= 6 → F	
senao → escreval("Aluno reprovado!")	4	5	4,5		Aluno reprovado!

Ex 10 — algoritmo "Positivo, negativo ou zero"

```
algoritmo "Positivo, negativo ou zero"

var
  n1: real

inicio

  escreval("Entre com um número")
  leia(n1)
  se n1 = 0 entao
    escreval(n1, " é igual a zero")
  fimse
  se n1 > 0 entao
    escreval(n1, " é positivo")
  fimse
  se n1 < 0 entao
    escreval(n1, " é negativo")
  fimse

finalgoritmo
```

Teste de mesa 1 — entradas: n1 = 0

Instrução	n1	Condição	Saída
escreval("Entre com um número")	—		Entre com um número
leia(n1)	0		
se n1 = 0 entao	0	0 = 0 → V	
escreval(n1, " é igual a zero")	0		0 é igual a zero
se n1 > 0 entao	0	0 > 0 → F	
se n1 < 0 entao	0	0 < 0 → F	

Teste de mesa 2 — entradas: n1 = 5,5

Instrução	n1	Condição	Saída
escreval("Entre com um número")	—		Entre com um número
leia(n1)	5,5		
se n1 = 0 entao	5,5	5,5 = 0 → F	
se n1 > 0 entao	5,5	5,5 > 0 → V	
escreval(n1, " é positivo")	5,5		5,5 é positivo
se n1 < 0 entao	5,5	5,5 < 0 → F	

Teste de mesa 3 — entradas: n1 = -12

Instrução	n1	Condição	Saída
escreval("Entre com um número")	—		Entre com um número
leia(n1)	-12		
se n1 = 0 entao	-12	-12 = 0 → F	
se n1 > 0 entao	-12	-12 > 0 → F	
se n1 < 0 entao	-12	-12 < 0 → V	
escreval(n1, " é negativo")	-12		-12 é negativo

Ex 11 — algoritmo "maior ou menor"

```
algoritmo "maior ou menor"

var
    n1, n2: real

inicio

    escreval("Entre com um número")
    leia(n1)
    escreval("Entre com um número")
    leia(n2)
    se n1 = n2 entao
        escreval(n1, " é igual a ", n2)
    senao
        se n1 < n2 entao
            escreval(n1, " é menor que ", n2)
        senao
            escreval(n1, " é maior que ", n2)
    fimse
fimse
finalgoritmo
```

Teste de mesa 1 — entradas: n1 = 7, n2 = 7

Instrução	n1	n2	Condição	Saída
escreval("Entre com um número")	—	—		Entre com um número
leia(n1)	7	—		
escreval("Entre com um número")	7	—		Entre com um número
leia(n2)	7	7		
se n1 = n2 entao	7	7	7 = 7 → V	
escreval(n1, " é igual a ", n2)	7	7		7 é igual a 7

Teste de mesa 2 — entradas: n1 = 3, n2 = 8

Instrução	n1	n2	Condição	Saída
escreval("Entre com um número")	—	—		Entre com um número
leia(n1)	3	—		
escreval("Entre com um número")	3	—		Entre com um número
leia(n2)	3	8		
se n1 = n2 entao	3	8	3 = 8 → F	
senao → se n1 < n2 entao	3	8	3 < 8 → V	
escreval(n1, " é menor que ", n2)	3	8		3 é menor que 8

Teste de mesa 3 — entradas: n1 = 10, n2 = -2,5

Instrução	n1	n2	Condição	Saída
escreval("Entre com um número")	—	—		Entre com um número
leia(n1)	10	—		
escreval("Entre com um número")	10	—		Entre com um número
leia(n2)	10	-2,5		
se n1 = n2 entao	10	-2,5	10 = -2,5 → F	
senao → se n1 < n2 entao	10	-2,5	10 < -2,5 → F	
senao → escreval(n1, " é maior que ",n2)	10	-2,5		10 é maior que -2,5

Ex 12 — algoritmo "Maior de idade"

```
algoritmo "Maior de idade"

var
  idade: inteiro

inicio
  escreval("Entre sua idade")
  leia(idade)
  se idade >= 18 entao
    escreval("Você é maior de idade")
  senao
    escreval("Você é menor de idade")
  fimse
fimalgoritmo
```

Teste de mesa 1 — entradas: idade = 18

Instrução	idade	Condição	Saída
escreval("Entre sua idade")	—		Entre sua idade
leia(idade)	18		
se idade >= 18 entao	18	18 >= 18 → V	
escreval("Você é maior de idade")	18		Você é maior de idade

Teste de mesa 2 — entradas: idade = 17

Instrução	idade	Condição	Saída
escreval("Entre sua idade")	—		Entre sua idade
leia(idade)	17		
se idade >= 18 entao	17	17 >= 18 → F	
senao → escreval("Você é menor de idade")	17		Você é menor de idade

Ex 13 — algoritmo "Faixas de nota"

```
algoritmo "Faixas de nota"

var
    nota: real

inicio
    escreval("Entre a nota: ")
    leia(nota)
    se nota = 6 entao
        escreval("Nota igual a média")
    senao
        se nota < 6 entao
            escreval("Nota abaixo da média")
        senao
            escreval("Nota acima da média")
        fimse
    fimse
fimse

finalgoritmo
```

Teste de mesa 1 — entradas: nota = 6

Instrução	nota	Condição	Saída
escreval("Entre a nota: ")	—		Entre a nota:
leia(nota)	6		
se nota = 6 entao	6	6 = 6 → V	
escreval("Nota igual a média")	6		Nota igual a média

Teste de mesa 2 — entradas: nota = 4

Instrução	nota	Condição	Saída
escreval("Entre a nota: ")	—		Entre a nota:
leia(nota)	4		
se nota = 6 entao	4	4 = 6 → F	
senao → se nota < 6 entao	4	4 < 6 → V	
escreval("Nota abaixo da média")	4		Nota abaixo da média

Teste de mesa 3 — entradas: nota = 8,5

Instrução	nota	Condição	Saída
escreval("Entre a nota: ")	—		Entre a nota:
leia(nota)	8,5		
se nota = 6 entao	8,5	8,5 = 6 → F	
senao → se nota < 6 entao	8,5	8,5 < 6 → F	
senao → escreval("Nota acima da média")	8,5		Nota acima da média

Ex 14 — algoritmo "Aprovação"

```
algoritmo "Aprovação"

var
    nota, frequencia: real

inicio
    escreval("Entre a nota: ")
    leia(nota)
    escreval("Entre a frequencia")
    leia(frequencia)
    se nota >= 6 e frequencia >= 75 entao
        escreval("Aluno Aprovado")
    senao
        escreval("Aluno reprovado")
    fimse
fimse
finalgoritmo
```

Teste de mesa 1 — entradas: nota = 6, frequencia = 75 — caso V e V

Instrução	nota	frequencia	nota >= 6	frequencia >= 75	Condição	Saída
escreval("Entre a nota: ")	—	—				Entre a nota:
leia(nota)	6	—				
escreval("Entre a frequencia")	6	—				Entre a frequencia
leia(frequencia)	6	75				
se nota >= 6 e frequencia >= 75 entao	6	75	V	V	V e V → V	
escreval("Aluno Aprovado")	6	75				Aluno Aprovado

Teste de mesa 2 — entradas: nota = 8, frequencia = 74 — caso V e F

Instrução	nota	frequencia	nota >= 6	frequencia >= 75	Condição	Saída
escreval("Entre a nota: ")	—	—				Entre a nota:
leia(nota)	8	—				
escreval("Entre a frequencia")	8	—				Entre a frequencia
leia(frequencia)	8	74				
se nota >= 6 e frequencia >= 75 entao	8	74	V	F	V e F → F	
senao → escreval("Aluno reprovado")	8	74				Aluno reprovado

Teste de mesa 3 — entradas: nota = 5, frequencia = 90 — caso F e V

Instrução	nota	frequencia	nota >= 6	frequencia >= 75	Condição	Saída
escreval("Entre a nota: ")	—	—				Entre a nota:
leia(nota)	5	—				
escreval("Entre a frequencia")	5	—				Entre a frequencia
leia(frequencia)	5	90				
se nota >= 6 e frequencia >= 75 entao	5	90	F	V	F e V → F	
senao → escreval("Aluno reprovado")	5	90				Aluno reprovado

Teste de mesa 4 — entradas: nota = 3, frequencia = 40 — caso F e F

Instrução	nota	frequencia	nota >= 6	frequencia >= 75	Condição	Saída
escreval("Entre a nota: ")	—	—				Entre a nota:
leia(nota)	3	—				
escreval("Entre a frequencia")	3	—				Entre a frequencia
leia(frequencia)	3	40				
se nota >= 6 e frequencia >= 75 entao	3	40	F	F	F e F → F	
senao → escreval("Aluno reprovado")	3	40				Aluno reprovado

Ex 15 — algoritmo "Desconto"

```

algoritmo "Desconto"

var
    valor: real
    vip: inteiro
inicio
    escreval("Entre o valor da compra")
    leia(valor)
    escreval("Entre se o cliente é VIP (1 para VIP ou 0 para não VIP)")
    leia(vip)
    se valor >= 100 ou vip = 1 entao
        escreval("Cliente com direito a desconto")
    senao
        escreval("Cliente sem direito a desconto")
    fimse
fim algoritmo

```

Teste de mesa 1 — entradas: valor = 100, vip = 1 — caso V ou V

Instrução	valor	vip	valor >= 100	vip = 1	Condição	Saída
escreval("Entre o valor da compra")	—	—				Entre o valor da compra
leia(valor)	100	—				
escreval("Entre se o cliente é VIP (1 ... ou 0 ...)")	100	—				Entre se o cliente é VIP (1 para VIP ou 0 para não VIP)
leia(vip)	100	1				
se valor >= 100 ou vip = 1 entao	100	1	V	V	V ou V → V	
escreval("Cliente com direito a desconto")	100	1				Cliente com direito a desconto

Teste de mesa 2 — entradas: valor = 250, vip = 0 — caso V ou F

Instrução	valor	vip	valor >= 100	vip = 1	Condição	Saída
escreval("Entre o valor da compra")	—	—				Entre o valor da compra
leia(valor)	250	—				
escreval("Entre se o cliente é VIP (1 ... ou 0 ...)")	250	—				Entre se o cliente é VIP (1 para VIP ou 0 para não VIP)
leia(vip)	250	0				
se valor >=100 ou vip = 1 entao	250	0	V	F	V ou F → V	
escreval("Cliente com direito a desconto")	250	0				Cliente com direito a desconto

Teste de mesa 3 — entradas: valor = 50, vip = 1 — caso F ou V

Instrução	valor	vip	valor >= 100	vip = 1	Condição	Saída
escreval("Entre o valor da compra")	—	—				Entre o valor da compra
leia(valor)	50	—				
escreval("Entre se o cliente é VIP (1 ... ou 0 ...)")	50	—				Entre se o cliente é VIP (1 para VIP ou 0 para não VIP)
leia(vip)	50	1				
se valor >=100 ou vip = 1 entao	50	1	F	V	F ou V → V	
escreval("Cliente com direito a desconto")	50	1				Cliente com direito a desconto

Teste de mesa 4 — entradas: valor = 50, vip = 0 — caso F ou F

Instrução	valor	vip	valor >= 100	vip = 1	Condição	Saída
escreval("Entre o valor da compra")	—	—				Entre o valor da compra
leia(valor)	50	—				
escreval("Entre se o cliente é VIP (1 ... ou 0 ...)")	50	—				Entre se o cliente é VIP (1 para VIP ou 0 para não VIP)
leia(vip)	50	0				
se valor >=100 ou vip = 1 entao	50	0	F	F	F ou F → F	
senao → escreval("Cliente sem direito a desconto")	50	0				Cliente sem direito a desconto

Ex 16 — algoritmo "Login"

```
algoritmo "Login"

var
  login, senha, testLog, testSen: inteiro
  bloq:logico
inicio
  escreval("Entre seu login")
  leia(login)
  escreval("Entre sua senha")
  leia(senha)
  escreval("O usuário está bloqueado? (digite VERDADEIRO ou FALSO)")
  leia(bloq)
  testLog <- 123
  testSen<- 456
  se login = testLog e senha = testSen e bloq = verdadeiro entao
    escreval("User bloqueado")
  senao
    se login = testLog e senha = testSen e bloq = falso entao
      escreval("Login Concluido")
    senao
      se login = testLog e senha <> testSen e bloq = verdadeiro entao
        escreval("User bloqueado e senha invalida")
      senao
        se login = testLog e senha <> testSen e bloq = falso entao
          escreval("Senha incorreta")
        senao
          se login <> testLog e senha = testSen e bloq = verdadeiro entao
            escreval("User bloqueado e login errado")
          senao
            se login <> testLog e senha = testSen e bloq = falso entao
              escreval("Login Invalido")
            senao
              escreval("Login Invalido")
            fimse
          fimse
        fimse
      fimse
    fimse
  fimse
  fimse
  fimse
  fimse
  fimse

finalgoritmo
```

Teste de mesa 1 — entradas: login = 123, senha = 456, bloq = falso

Instrução	login	senha	testLog	testSen	bloq	Condição	Saída
escreval("Entre seu login")	—	—	—	—	—		Entre seu login
leia(login)	123	—	—	—	—		
escreval("Entre sua senha")	123	—	—	—	—		Entre sua senha
leia(senha)	123	456	—	—	—		
escreval("O usuário está bloqueado? ...")	123	456	—	—	—		O usuário está bloqueado? (digite VERDADEIRO ou FALSO)
leia(bloq)	123	456	—	—	falso		
testLog <- 123	123	456	123	—	falso		
testSen <- 456	123	456	123	456	falso		
se login = testLog e senha = testSen e bloq = verdadeiro	123	456	123	456	falso	V e V e F → F	
se login = testLog e senha = testSen e bloq = falso	123	456	123	456	falso	V e V e V → V	
escreval("Login Concluido")	123	456	123	456	falso		Login Concluido

Teste de mesa 2 — entradas: login = 123, senha = 456, bloq = verdadeiro

Instrução	login	senha	testLog	testSen	bloq	Condição	Saída
escreval("Entre seu login")	—	—	—	—	—		Entre seu login
leia(login)	123	—	—	—	—		
escreval("Entre sua senha")	123	—	—	—	—		Entre sua senha
leia(senha)	123	456	—	—	—		
escreval("O usuário está bloqueado? ...")	123	456	—	—	—		O usuário está bloqueado? (digite VERDADEIRO ou FALSO)
leia(bloq)	123	456	—	—	verdadeiro		
testLog <- 123	123	456	123	—	verdadeiro		
testSen <- 456	123	456	123	456	verdadeiro		
se login = testLog e senha = testSen e bloq = verdadeiro	123	456	123	456	verdadeiro	V e V e V → V	
escreval("User bloqueado")	123	456	123	456	verdadeiro		User bloqueado

Teste de mesa 3 — entradas: login = 123, senha = 111, bloq = falso

Instrução	login	senha	testLog	testSen	bloq	Condição	Saída
escreval("Entre seu login")	—	—	—	—	—		Entre seu login
leia(login)	123	—	—	—	—		
escreval("Entre sua senha")	123	—	—	—	—		Entre sua senha
leia(senha)	123	111	—	—	—		
escreval("O usuário está bloqueado? ...")	123	111	—	—	—		O usuário está bloqueado? (digite VERDADEIRO ou FALSO)
leia(bloq)	123	111	—	—	falso		
testLog <- 123	123	111	123	—	falso		
testSen <- 456	123	111	123	456	falso		
se login = testLog e senha = testSen e bloq = verdadeiro	123	111	123	456	falso	V e F e F → F	
se login = testLog e senha = testSen e bloq = falso	123	111	123	456	falso	V e F e V → F	
se login = testLog e senha <> testSen e bloq = verdadeiro	123	111	123	456	falso	V e V e F → F	
se login = testLog e senha <> testSen e bloq = falso	123	111	123	456	falso	V e V e V → V	
escreval("Senha incorreta")	123	111	123	456	falso		Senha incorreta

Teste de mesa 4 — entradas: login = 123, senha = 111, bloq = verdadeiro

Instrução	login	senha	testLog	testSen	bloq	Condição	Saída
escreval("Entre seu login")	—	—	—	—	—		Entre seu login
leia(login)	123	—	—	—	—		
escreval("Entre sua senha")	123	—	—	—	—		Entre sua senha
leia(senha)	123	111	—	—	—		
escreval("O usuário está bloqueado? ...")	123	111	—	—	—		O usuário está bloqueado? (digite VERDADEIRO ou FALSO)
leia(bloq)	123	111	—	—	verdadeiro		
testLog <- 123	123	111	123	—	verdadeiro		
testSen <- 456	123	111	123	456	verdadeiro		
se login = testLog e senha = testSen e bloq = verdadeiro	123	111	123	456	verdadeiro	V e F e V → F	
se login = testLog e senha = testSen e bloq = falso	123	111	123	456	verdadeiro	V e F e F → F	
se login = testLog e senha <> testSen e bloq = verdadeiro	123	111	123	456	verdadeiro	V e V e V → V	
escreval("User bloqueado e senha invalida")	123	111	123	456	verdadeiro		User bloqueado e senha invalida

Teste de mesa 5 — entradas: login = 999, senha = 456, bloq = falso

Instrução	login	senha	testLog	testSen	bloq	Condição	Saída
escreval("Entre seu login")	—	—	—	—	—		Entre seu login
leia(login)	999	—	—	—	—		
escreval("Entre sua senha")	999	—	—	—	—		Entre sua senha
leia(senha)	999	456	—	—	—		
escreval("O usuário está bloqueado? ...")	999	456	—	—	—		O usuário está bloqueado? (digite VERDADEIRO ou FALSO)
leia(bloq)	999	456	—	—	falso		
testLog <- 123	999	456	123	—	falso		
testSen <- 456	999	456	123	456	falso		
se login = testLog e senha = testSen e bloq = verdadeiro	999	456	123	456	falso	F e V e F → F	
se login = testLog e senha = testSen e bloq = falso	999	456	123	456	falso	F e V e V → F	
se login = testLog e senha <> testSen e bloq = verdadeiro	999	456	123	456	falso	F e F e F → F	
se login = testLog e senha <> testSen e bloq = falso	999	456	123	456	falso	F e F e V → F	
se login <> testLog e senha = testSen e bloq = verdadeiro	999	456	123	456	falso	V e V e F → F	
se login <> testLog e senha = testSen e bloq = falso	999	456	123	456	falso	V e V e V → V	
escreval("Login Invalido")	999	456	123	456	falso		Login Invalido

Teste de mesa 6 — entradas: login = 999, senha = 456, bloq = verdadeiro

Instrução	login	senha	testLog	testSen	bloq	Condição	Saída
escreval("Entre seu login")	—	—	—	—	—		Entre seu login
leia(login)	999	—	—	—	—		
escreval("Entre sua senha")	999	—	—	—	—		Entre sua senha
leia(senha)	999	456	—	—	—		
escreval("O usuário está bloqueado? ...")	999	456	—	—	—		O usuário está bloqueado? (digite VERDADEIRO ou FALSO)
leia(bloq)	999	456	—	—	verdadeiro		
testLog <- 123	999	456	123	—	verdadeiro		
testSen <- 456	999	456	123	456	verdadeiro		
se login = testLog e senha = testSen e bloq = verdadeiro	999	456	123	456	verdadeiro	F e V e V → F	
se login = testLog e senha = testSen e bloq = falso	999	456	123	456	verdadeiro	F e V e F → F	
se login = testLog e senha <> testSen e bloq = verdadeiro	999	456	123	456	verdadeiro	F e F e V → F	
se login = testLog e senha <> testSen e bloq = falso	999	456	123	456	verdadeiro	F e F e F → F	
se login <> testLog e senha = testSen e bloq = verdadeiro	999	456	123	456	verdadeiro	V e V e V → V	
escreval("User bloqueado e login errado")	999	456	123	456	verdadeiro		User bloqueado e login errado

Teste de mesa 7 — entradas: login = 999, senha = 111, bloq = falso

Instrução	login	senha	testLog	testSen	bloq	Condição	Saída
escreval("Entre seu login")	—	—	—	—	—		Entre seu login
leia(login)	999	—	—	—	—		
escreval("Entre sua senha")	999	—	—	—	—		Entre sua senha
leia(senha)	999	111	—	—	—		
escreval("O usuário está bloqueado? ...")	999	111	—	—	—		O usuário está bloqueado? (digite VERDADEIRO ou FALSO)
leia(bloq)	999	111	—	—	falso		
testLog <- 123	999	111	123	—	falso		
testSen <- 456	999	111	123	456	falso		
se login = testLog e senha = testSen e bloq = verdadeiro	999	111	123	456	falso	F e F e F → F	
se login = testLog e senha = testSen e bloq = falso	999	111	123	456	falso	F e F e V → F	
se login = testLog e senha <> testSen e bloq = verdadeiro	999	111	123	456	falso	F e V e F → F	
se login = testLog e senha <> testSen e bloq = falso	999	111	123	456	falso	F e V e V → F	
se login <> testLog e senha = testSen e bloq = verdadeiro	999	111	123	456	falso	V e F e F → F	
se login <> testLog e senha = testSen e bloq = falso	999	111	123	456	falso	V e F e V → F	
senao → escreval("Login Invalido")	999	111	123	456	falso		Login Invalido